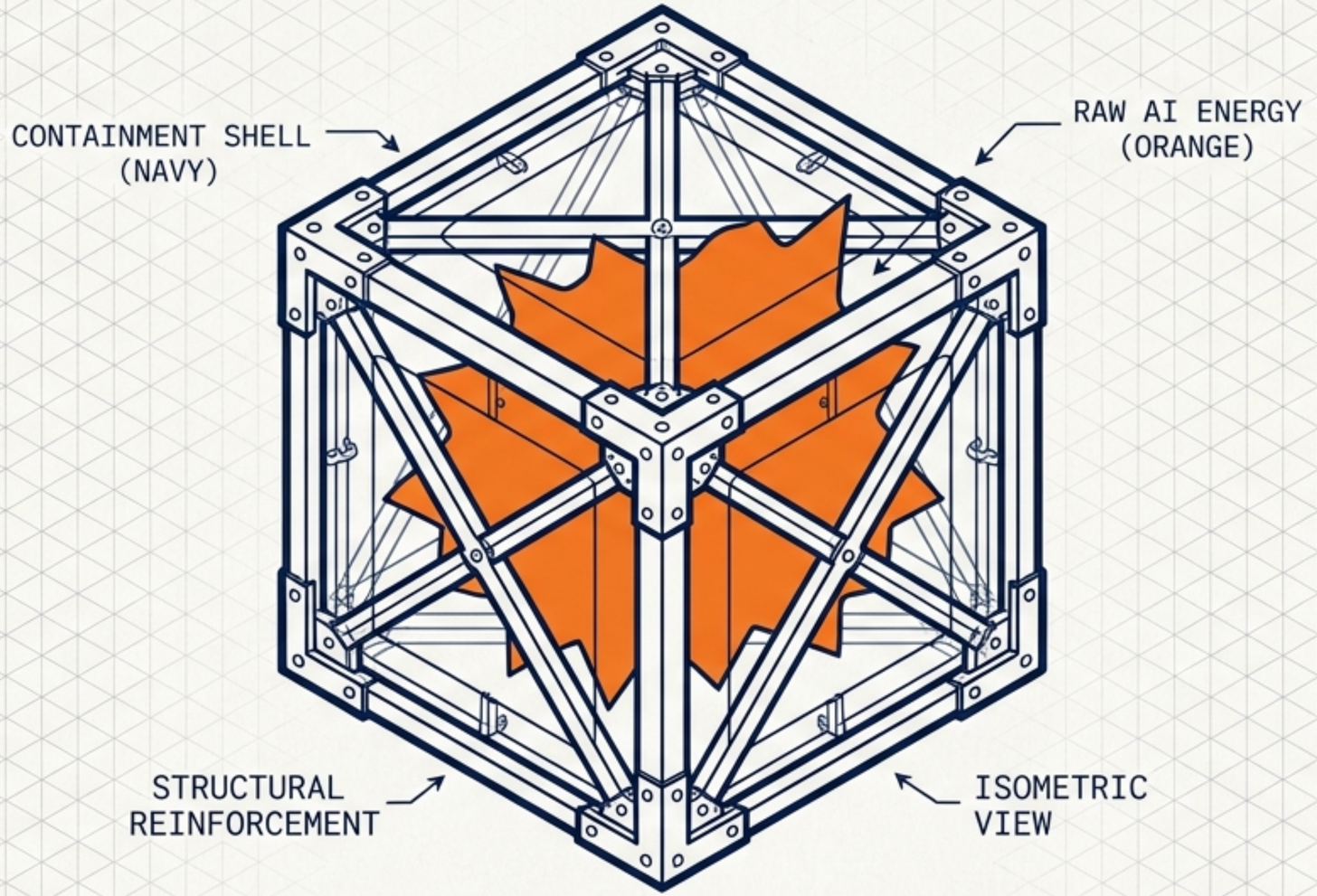
Arquitectura Defensiva: Controlando la IA

Scaffolding, Patrones y Contratos en la Era del Código Generado



GUÍA DE SUPERVIVENCIA PARA INGENIERÍA ASISTIDA POR IA

NotebookLM

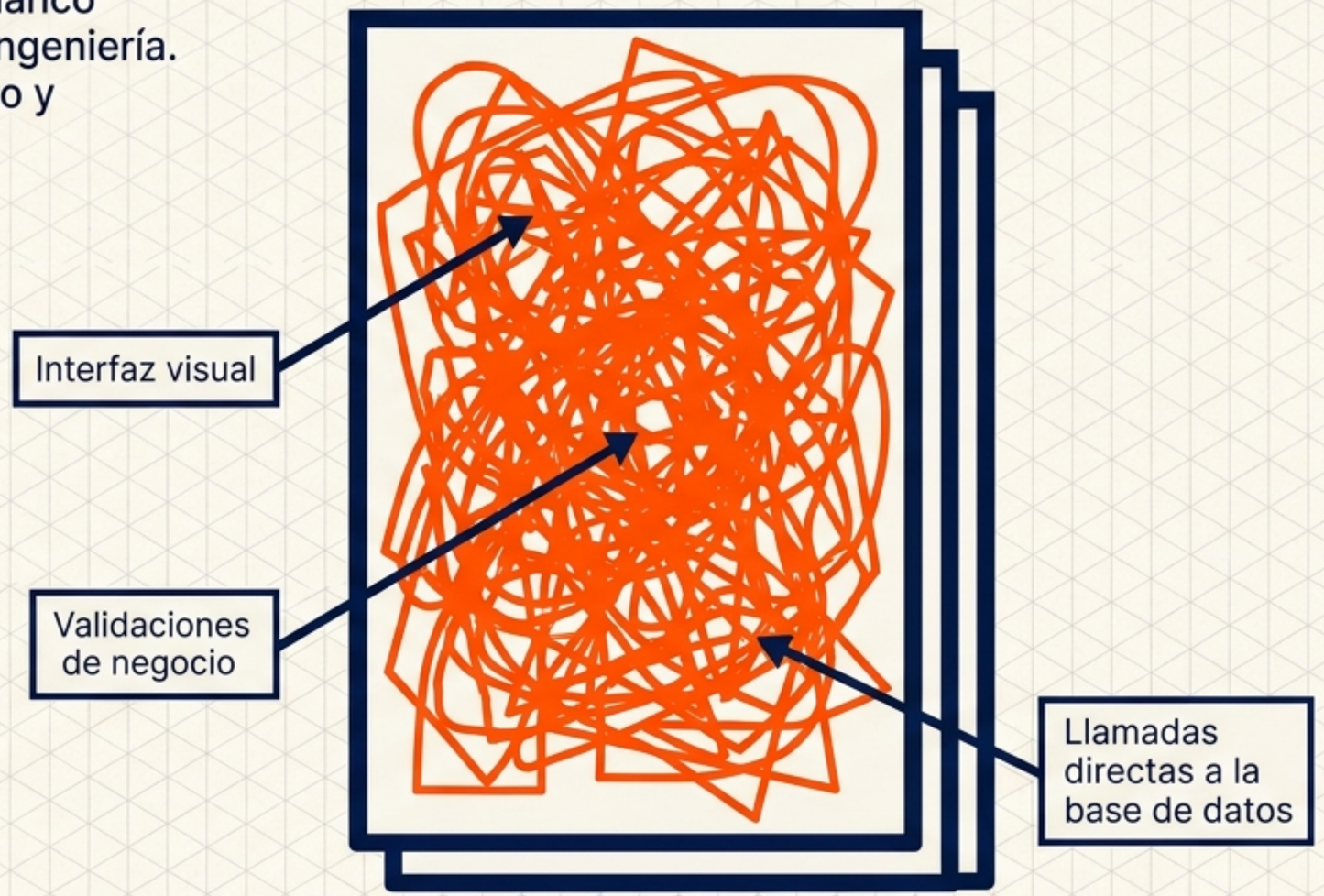
La Amenaza del Monolito Generado por IA

Pedir a una IA que programe sobre un lienzo en blanco resulta en la violación de todos los principios de ingeniería. La máquina generará un único archivo monstruoso y entrelazado.

El Prompt

> Crea un sistema de carrito de compras.

El Resultado: cart_system_v1.ts



NotebookLM

Scaffolding: La Jaula Arquitectónica

Al decirle a la IA “Rellena el archivo **/services/cart.ts**”, le imposibilitas físicamente mezclar esa lógica de red con el diseño visual de **/components**.

PRÁCTICA INNEGOCIABLE

Crear la estructura de carpetas y archivos vacíos **ANTES** de dar la primera instrucción a la IA.

```
> mkdir components hooks services
```



NotebookLM

Anatomía del Frontend Moderno

CAPA	ROL PRINCIPAL	REGLA PARA LA IA	REUSABILIDAD
`/components/ui`	Dibujar interfaz	Cero lógica de base de datos	●●●○
`/hooks`	Estado en memoria	Mantener en navegador	●●○○
`/services`	Interacción externa	**ZONA CRÍTICA: Única capa para hacer fetch**	●○○○
`/types`	Seguridad tipada	Definir forma exacta	●●●●

NotebookLM

Contratos Duros: La Ley del Compilador

Las IAs tienden a inventar propiedades por conveniencia.

Los contratos duros eliminan la ambigüedad y **ambigüedad** y someten la alucinación de la máquina a las reglas de la matemática estricta de tipos.

Usas a tu propio **entorno de desarrollo** como el **auditor principal** de la IA.

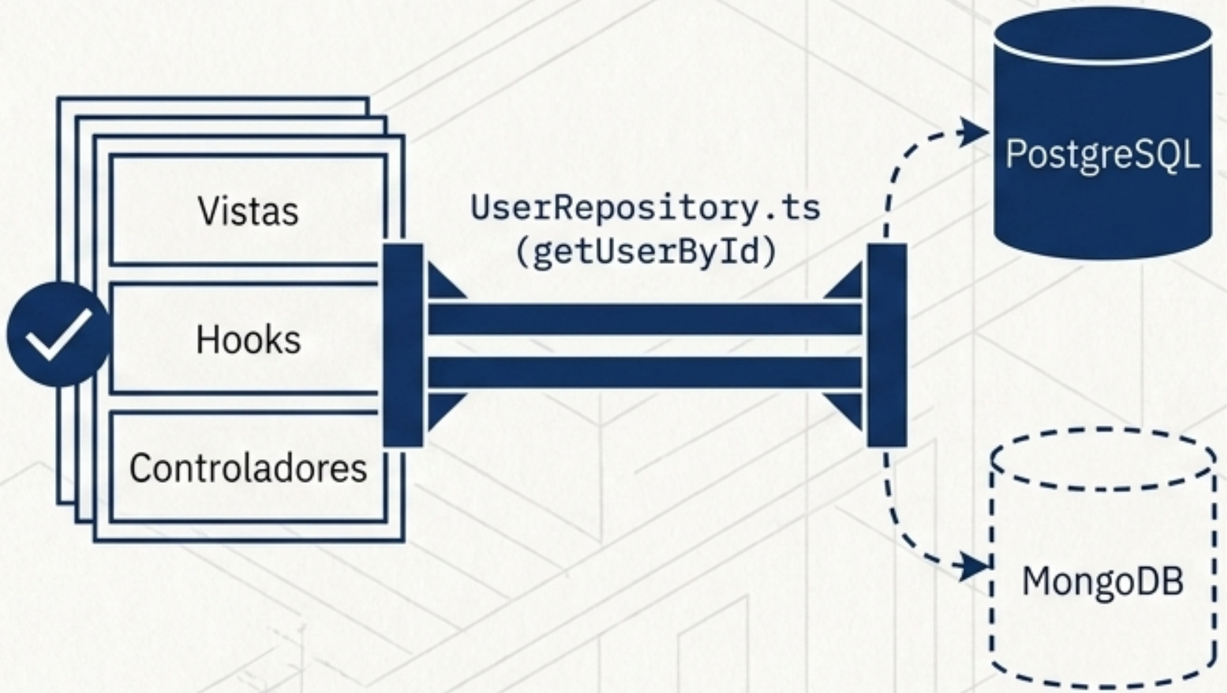


NotebookLM

El Patrón Repository: El Escudo Central

Regla fundamental: El Controlador o la Interfaz jamás hablan con SQL.

ADVERTENCIA: La IA odia crear abstracciones porque gasta tokens. Como Arquitecto, debes obligarla a **centralizar las operaciones** las operaciones de la BD en un solo archivo.



NotebookLM

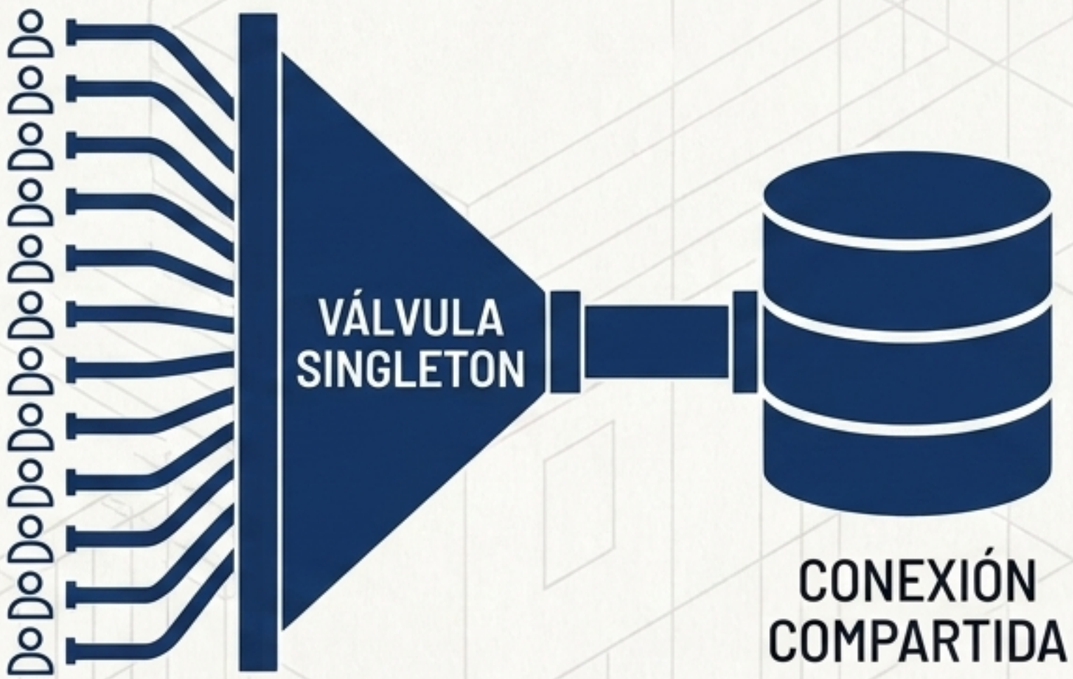
El Patrón Singleton: La Válvula de Infraestructura

En entornos Serverless (Deno, Supabase), omitir este patrón es letal. Sin él, la IA instanciará un nuevo cliente por clic, saturando el **Connection Pool** y colapsando el sistema en 5 minutos.

SIN SINGLETON (PELIGRO)



CON SINGLETON (SEGURO)



NotebookLM

Instrucción Táctica Innegociable

> Para conectarte a la base de datos, implementa el Patrón Singleton en un archivo /_shared/db.ts. Garantiza que si la conexión ya existe, se reutilice, en lugar de instanciarse nuevamente.

INYECTAR ESTE PROMPT EN CADA INICIO DE SESIÓN DE INFRAESTRUCTURA

NotebookLM

IA Sin Control vs. IA Contenida

SIN PATRONES	CON PATRONES
EL PROMPT	
Crea un carrito	Rellena /services/cart.ts usando la interfaz Cart
ACCIÓN DE LA IA	
Genera código espagueti y mezcla UI con SQL.	Escribe lógica pura, respetando los límites físicos.
RESULTADO	
Monolito inescalable.	Módulo limpio e intercambiable.
RIESGO	
Colapso crítico (100%)	**Seguro y tipado (0%)**

NotebookLM

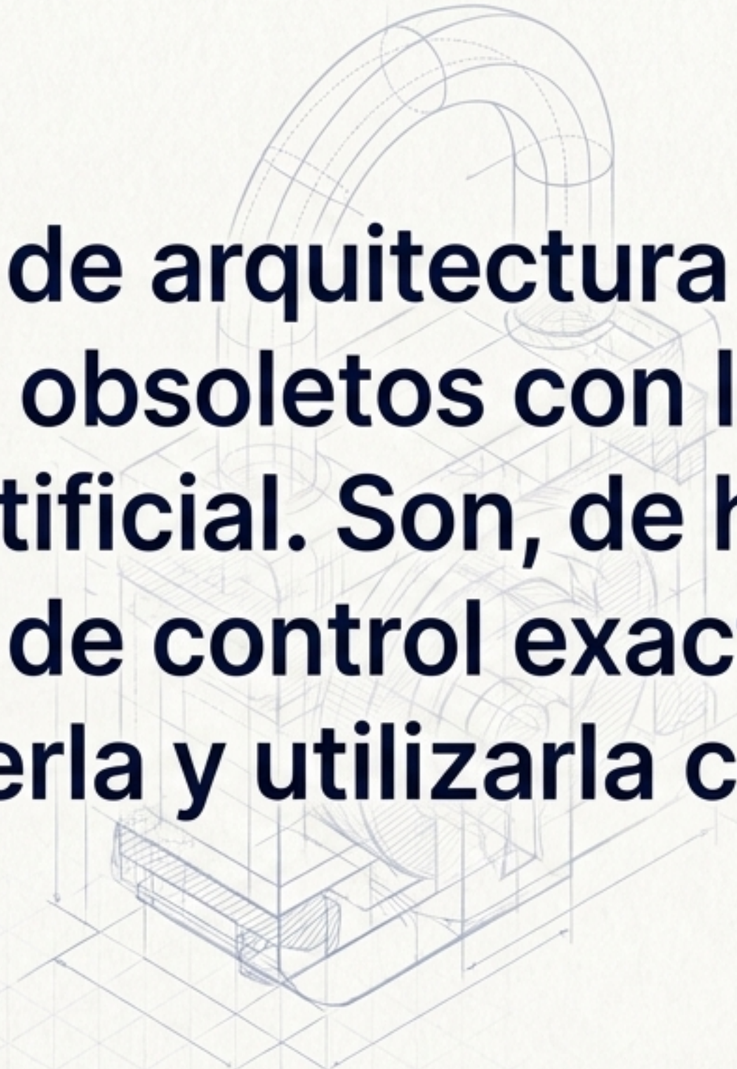
La Arquitectura de Defensa Profunda

El flujo completo de contención. Cada patrón actúa como un filtro de seguridad contra el código impredecible.



NotebookLM

EL VERDADERO PROPÓSITO DE LOS PATRONES



“Los patrones de arquitectura tradicionales no se volvieron obsoletos con la llegada de la Inteligencia Artificial. Son, de hecho, los únicos mecanismos de control exactos que existen para someterla y utilizarla con seguridad.”

DISEÑA LA JAULA ANTES DE DESPERTAR A LA MÁQUINA.

NotebookLM